

A PROJECT REPORT ON

Design of AXI4-Lite Crossbar Interconnect

SUBMITTED IN PARTIAL FULFILMENT
OF
THE REQUIREMENTS OF
THE AWARDS OF THE

PG-DIPLOMA IN VERY LARGE SCALE INTEGRATED CIRCUITS DESIGN

**OFFERED BY
C-DAC HYDERABAD**

BY

- | | |
|-------------------------|-------------------|
| 1. AMAN CHOURASIA | PRN: 250850333001 |
| 2. OMKAR DHANAJI BODAKE | PRN: 250850333022 |
| 3. SAI VILAS DESALE | PRN: 250850333028 |
| 4. ARAVA PAVAN SRI SAI | PRN: 250850333004 |
| 5. ARYAN SINGH RANA | PRN: 250850333005 |

**ADVANCED COMPUTING TRAINING
SCHOOL
C-DAC
HYDERABAD-501510**

AUGUST 2025



CERTIFICATE

This is to certify that this is a bonafide record of project entitled **“Design of AXI4-Lite Crossbar Interconnect”**. Aman Chourasia: 250850333001, Omkar Dhanaji Bodake: 250850333022, Sai Vilas Desale: 250850333028, Arava Pavan Sri Sai: 250850333004, Aryan Singh Rana: 250850333005 have completed project work as part of **Diploma in Very Large Scale Integrated Circuits (August 2025 Batch)**, a PG course offered by C-DAC Hyderabad. They have completed project work under the supervision of **Mr. Praveen Verma & Ms. Keerthi Dubba**. Their performance has found to be good.

Mr. Praveen Verma & Ms. Keerthi Dubba

(_____)

DATE :

PLACE : C-DAC, HARDWARE PARK, HYDERABAD.

ACKNOWLEDGEMENT

This project “*Design of AXI4-Lite Crossbar Interconnect*” was a great learning experience for us and we are submitting this work to Advanced Computing Training School (CDAC Hyderabad). We are very glad to mention the name of ***Mr. Praveen Verma & Ms. Keerthi Dubba*** for their valuable guidance to work on this project. Their guidance and support helped me to overcome various obstacles and intricacies during the course of project work.

Our heartfelt thanks goes to ***Mr. Shivaprasad Sunka*** (Course Co-ordinator, PG-DVLSI) who gave all the required support and kind coordination to provide all the necessities and extra hours to complete the project and throughout the course up to the last day here in C-DAC , Hyderabad.

(PG-DVLSI AUG 2025)

1. Aman Chourasia: 250850333001
2. Omkar Dhanaji Bodake: 250850333022
3. Sai Vilas Desale: 250850333028
4. Arava Pavan Sri Sai: 250850333004
5. Aryan Singh Rana: 250850333005

Table Of Contents

Topic	Page
Abstract	1
1. Introduction	2
1.1 Problem Statement	2
1.2 Objectives	3
2. Literature Survey	4
3. System Specifications	7
3.1 System Overview	7
3.2 Functional Blocks Of System	7
3.3 Specifications	8
4. Block Diagram	9
5. Features Of AXI4-Lite Crossbar Interconnect	11
6. Methodology	14
7. Working	19
8. List Of Signals	23
9. Waveform & Simulation	26
10. Future Scope	29
11. Conclusion	30
12. References	31

ABSTRACT

Modern SoC designs require efficient communication between multiple IP blocks using standard bus protocols. AXI-Lite is a widely used AMBA protocol for low-latency control and configuration of peripherals. This project presents the design of an AXI-Lite Crossbar Interconnect that enables communication between multiple masters and multiple slave peripherals. Address-based decoding and arbitration logic ensure correct transaction routing and fair access among masters. The design strictly follows AXI-Lite handshaking and supports back-pressure handling. The proposed architecture is modular and scalable, reflecting industry-relevant frontend RTL design practices.

1. Introduction

Modern SoC (System-on-Chip) designs contain multiple processing blocks such as processors, peripherals, and memory modules that must communicate efficiently. As the number of modules increases, direct point-to-point connections become complex, slow, and difficult to manage. To solve this, an interconnect architecture is required that allows multiple masters to access multiple slaves in an organized and conflict-free manner.

The AXI4-Lite Crossbar Interconnect is a communication system that connects several master devices to multiple slave devices using the AXI protocol. It ensures correct data transfer, address routing, arbitration between masters, and control of read/write operations while following AXI handshaking rules. This design improves scalability, reliability, and performance of digital systems used in embedded and FPGA-based applications.

1.1 Problem Statement

- Multiple AXI4-Lite masters need access to multiple slave peripherals
- Direct connections increase design complexity and wiring overhead
- Simultaneous master requests lead to contention and access conflicts
- Lack of proper arbitration can cause data inconsistency
- Existing simple interconnects are not scalable
- Need for a standardized AXI4-Lite based crossbar solution

1.2 Objectives

- Design and implementation of an AXI4-Lite Crossbar Interconnect enabling communication between multiple masters and multiple slaves.
- Address decoding mechanism that routes each transaction to the correct slave based on the address.
- Arbitration logic that resolves conflicts when multiple masters request access to the same slave.
- Control logic that manages AXI read and write channels while maintaining proper VALID–READY handshaking.
- Integration of skid buffers to handle stalls and prevent data loss during backpressure.

2. Literature Survey

2.1 AMBA and AXI Protocols

The AMBA (Advanced Microcontroller Bus Architecture) specification developed by ARM defines standard interfaces for SoC communication. AXI (Advanced eXtensible Interface), part of AMBA 4, supports high performance and flexible interconnects through independent read and write channels, pipelined transactions, and multiple outstanding operations.

Key Reference:

ARM Ltd. (2010). AMBA® AXI and ACE Protocol Specification v1.0. ARM Holdings.

This specification forms the foundation of AXI4 and AXI4-Lite protocols and is the primary source for understanding handshaking, channel structure, and timing.

2.2 AXI4-Lite Control Interfaces

Several researchers note that while full AXI4 supports high-bandwidth burst transfers, AXI4-Lite is specifically optimized for control and configuration path communication in SoCs.

Relevant Papers:

1) Haoyu Liu, Jingfang Huang, Xuehai Qian (2020). Design and Implementation of AXI4-Lite Based Control Subsystem for FPGA Applications. IEEE International Symposium on FPGA.

This work discusses the use of AXI4-Lite for control interfaces in FPGA blocks and highlights its simplicity and protocol safety for register access.

2) Zhu, J., Wang, S., & Liu, M. (2018). Efficient Crossbar Interconnect Design for AXI4-Lite Control Interfaces. International Journal of Embedded Systems.

The authors implement crossbar logic optimized for AXI4-Lite, focusing on scalable decoding and arbitration mechanisms.

2.3 Crossbar Interconnect Architectures

Crossbars allow multiple masters to communicate with multiple slaves without contention on a shared bus. They provide parallel communication paths and avoid bottlenecks inherent in simple bus systems.

Key Papers:

1) Zhao, Y., & Wang, L. (2019). Performance Analysis of AMBA AXI Crossbar Interconnect. *Journal of Systems Architecture*.

This research compares shared buses with crossbar interconnects and quantifies improvements in latency and throughput.

2) Roth, R., & Gajski, D. (2017). High-Performance Crossbar Switches in SoC Interconnects. *ACM Transactions on Embedded Computing Systems*.

The authors explore arbitration strategies and how they impact system performance in multi-master environments.

2.4 Arbitration Mechanisms in Interconnects

Arbitration ensures that when multiple masters request the same slave, only one is granted access at a time. Several arbitration schemes have been proposed in literature.

Important Works:

1) R. Gupta & P. Mehrotra (2021). A Survey on Arbitration Techniques in NoC and SoC Interconnects. *International Journal of VLSI Design & Communication Systems (VLSICS)*.

This survey covers fixed priority, round-robin, and fairness-based arbitration schemes, providing a basis for selecting suitable logic for crossbars.

2) N. Jain & A. Kumar (2018). Round Robin Arbitration in Crossbar Interconnect for FPGA. IEEE Conference on Computer Design.

This paper specifically analyses round-robin arbitration to ensure fairness among requesters in FPGA interconnects.

3. System Specifications

3.1 System Overview

The proposed system is an AXI4-Lite Crossbar Interconnect designed to connect multiple AXI masters with multiple AXI slaves. The crossbar enables parallel communication paths, ensuring that masters can access different slaves simultaneously without bus contention. The design is optimized for control-path communication rather than high-bandwidth data transfer.

The system is implemented in RTL and verified through simulation. It includes arbitration logic, address decoding, buffering, and transaction tracking to ensure reliable protocol operation.

3.2 Functional Blocks of the System

The AXI4-Lite crossbar consists of the following internal modules:

- Skid Buffers: Temporary buffers placed at input channels to handle backpressure and prevent data loss when the downstream stage stalls.
- Address Decoder: Decodes incoming address signals and determines which slave should receive the transaction.
- Arbitration Logic: Resolves conflicts when multiple masters request access to the same slave.
- Control Logic: Manages channel switching, grant holding, and protocol handshaking.
- Pending Transaction Counters: Track the number of outstanding read and write transactions.
- Slave Output Logic: Routes valid signals, addresses, and data from selected master to slave.
Return Channel Logic: Routes response signals (B and R channels) from slave back to the correct master

3.3 Specifications

- Configuration: 2 Masters (NM=2), 2 Slaves (NS=2)
- Protocol: AXI4-Lite (5 Channels)
- Throughput Goal: 1 Transaction / Clock Cycle
- Latency: Minimum 2 cycles (Arbitration + Routing)
- Parameters:

C_AXI_DATA_WIDTH=32,

C_AXI_ADDR_WIDTH=32

4. Block Diagram

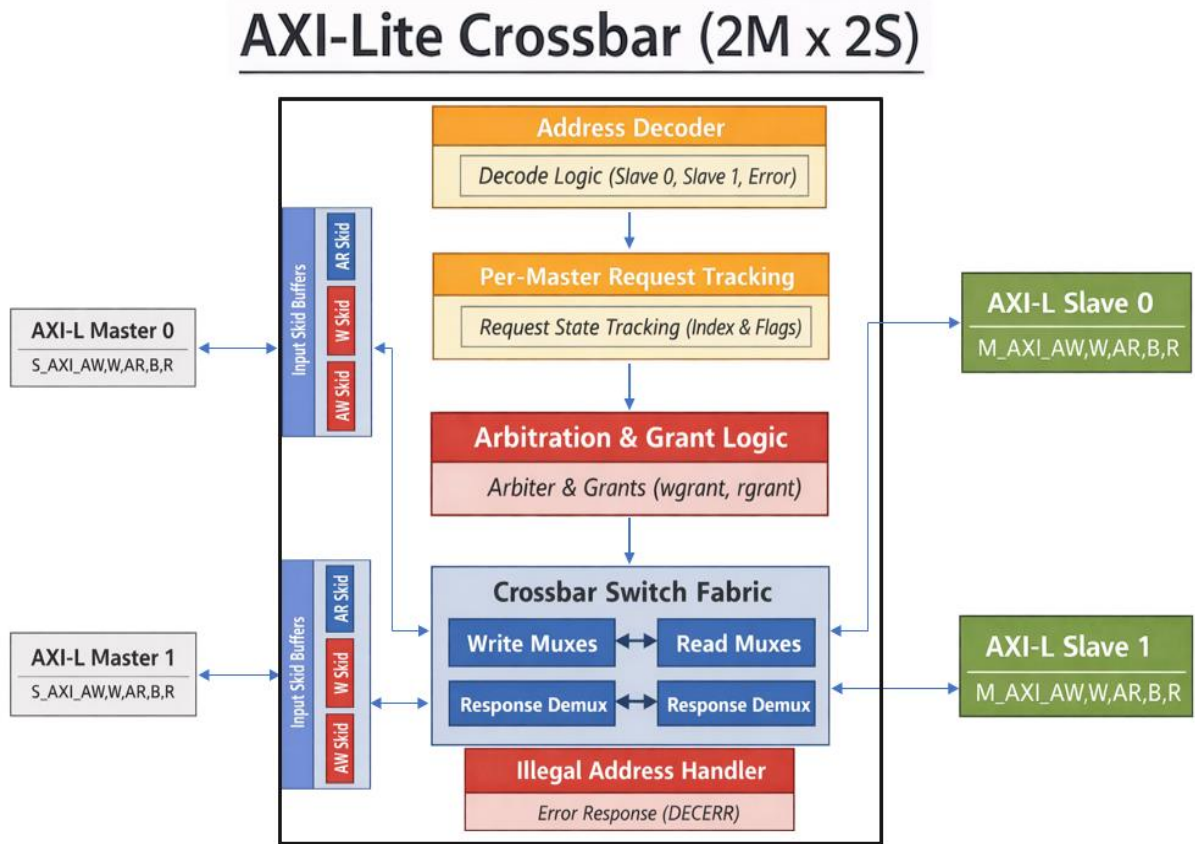


Fig 4.1: Crossbar block diagram.

The block diagram represents the internal architecture of the AXI-Lite Crossbar interconnect used in the proposed system. The crossbar enables communication between multiple AXI-Lite masters and multiple AXI-Lite slaves by providing address-based routing and controlled access to shared resources. This architecture improves scalability and allows efficient utilization of the AXI-Lite bus

The AXI-Lite Crossbar consists of master interfaces, input skid buffers, an address decoder, arbitration and grant logic, a switch fabric, and slave interfaces. Skid buffers are used to handle backpressure and to ensure stable data transfer

without creating combinational loops. The address decoder identifies the target slave for each transaction based on predefined address ranges.

When a master initiates a read or write operation, the request is first buffered and then decoded to determine the destination slave. If multiple masters request access to the same slave simultaneously, the arbitration logic selects one master according to the defined arbitration policy. The switch fabric establishes the connection between the granted master and the selected slave, allowing the transaction to proceed.

The block diagram also includes an illegal address handling mechanism to manage transactions directed to unmapped address regions. This ensures protocol compliance and reliable system operation. Overall, the AXI-Lite Crossbar architecture provides a modular, scalable, and efficient interconnection framework suitable for SoC-level designs.

5. Features of AXI4-Lite Crossbar Interconnect

The AXI4-Lite Crossbar Interconnect provides a structured and efficient communication system between multiple masters and slaves. Its features make it suitable for FPGA and SoC-based embedded systems.

5.1 Multi-Master Multi-Slave Communication

The crossbar allows multiple masters to communicate with multiple slaves simultaneously. Each master can independently access any slave, improving system performance and parallelism compared to single-bus architectures.

5.2 Address-Based Routing

Transactions are routed to slaves using address decoding logic. Each slave is assigned a specific address range, and the decoder ensures that every request reaches the correct slave without conflict.

5.3 Independent Read and Write Channels

AXI4-Lite uses separate channels for read and write operations:

- Write Address (AW)
- Write Data (W)
- Write Response (B)
- Read Address (AR)
- Read Data (R)

This separation allows concurrent read and write operations, increasing throughput.

5.4 Arbitration Mechanism

When two or more masters request access to the same slave, the crossbar uses arbitration logic to decide which master gets access. This prevents data collision and ensures fair resource sharing.

5.5 One Transaction at a Time per Channel

Since it is AXI4-Lite, each channel handles single-beat transactions only. This simplifies control logic and makes the design lightweight and easy to implement.

5.6 Skid Buffer Support

Skid buffers are included at input channels to:

- Handle backpressure
- Avoid data loss
- Maintain timing stability

5.7 Pending Transaction Tracking

The design includes counters that track outstanding read and write transactions. This ensures:

- No overflow
- Correct response matching
- Protocol compliance

5.8 Error Handling

If an address does not match any slave or a protocol issue occurs, the crossbar generates an interconnect error response, ensuring reliable operation.

5.9 Scalable Architecture

The number of masters and slaves can be increased by changing parameters. This makes the design reusable for different system sizes.

5.10 Protocol Compliance

The crossbar fully follows AXI4-Lite handshake rules:

- VALID/READY mechanism
- Separate channels
- Response signaling

6. Methodology

6.1 Requirement Analysis

The first stage of the project involved identifying the communication challenges in modern SoC systems. Multiple processing units and peripherals need to share data efficiently, but a direct point-to-point connection increases wiring complexity and hardware cost. Therefore, an interconnect that can route transactions between multiple masters and slaves using a standard protocol was required.

The system requirements were defined as:

- Support for multiple AXI4-Lite masters and slaves
- Address-based routing
- Conflict handling when multiple masters access the same slave
- Proper handshake management using VALID and READY signals
- Prevention of transaction loss under stall conditions
- Lightweight design suitable for FPGA implementation

These requirements formed the base of the crossbar architecture.

6.2 Study of AXI4-Lite Protocol

Before implementation, the AXI4-Lite protocol was studied in detail to ensure protocol compliance. AXI4-Lite is a simplified version of AXI used for low-bandwidth control registers. It supports only single-beat transactions and uses five independent channels:

- Write Address (AW) Channel
- Write Data (W) Channel
- Write Response (B) Channel
- Read Address (AR) Channel

- Read Data (R) Channel

Each channel uses the VALID/READY handshake mechanism. Understanding that these channels operate independently was crucial in designing control and arbitration logic.

6.3 System Architecture Planning

After protocol understanding, a modular architecture was planned. The crossbar was divided into functional blocks:

1. Skid Buffers
2. Address Decoder
3. Arbitration Logic
4. Control Logic
5. Pending Transaction Counters
6. Response Routing Logic

This modular approach simplified debugging, testing, and future scalability.

6.4 RTL Design Approach

The crossbar was implemented using Verilog RTL design. Parameterization was used so that the number of masters (NM) and slaves (NS) could be changed without redesigning the logic. Separate modules were developed for each functional block and later integrated.

Design practices followed:

- Synchronous design using a common clock
- Reset-based initialization
- Use of generate blocks for scalability
- Separation of combinational and sequential logic

6.5 Skid Buffer Design

Skid buffers were introduced at the input of AW, W, and AR channels. These buffers temporarily store transactions when the downstream slave is not ready.

Purpose:

- Handle backpressure
- Avoid data loss
- Break long combinational paths
- Improve timing performance

This ensures that transactions are safely held until the slave can accept them.

6.6 Address Decoder Implementation

Address decoding was implemented using base address and mask comparison. Each incoming address is checked against slave address ranges. If matched, a one-hot decode signal is generated to select the target slave.

If no address matches, an interconnect error response is generated. This block ensures correct routing of transactions.

6.7 Arbitration Logic Development

When multiple masters request access to the same slave, arbitration logic decides which master gets access. Grant signals are generated based on request availability and channel status.

The logic ensures:

- Only one master connects to a slave at a time
- No two masters conflict
- Fair channel switching

A linger mechanism was also implemented to avoid frequent switching when the channel becomes temporarily idle.

6.8 Control Logic Design

Control logic manages the overall flow of transactions. It coordinates between request signals, grant signals, and handshake signals. It tracks whether a channel should stay connected, switch to another slave, or release the connection.

This logic ensures correct timing and synchronization between different AXI channels.

6.9 Pending Transaction Counter

Counters were used to track outstanding transactions:

- `awpending` → pending write addresses
- `rpending` → pending read addresses
- `missing_wdata` → write data not yet received

These counters prevent overflow and help determine whether buffers are empty or full.

6.10 Integration of Modules

After individual blocks were verified, all modules were integrated into the crossbar top module. Signal routing between masters and slaves was connected according to grant and index signals.

6.11 Testbench Development

A testbench was written to simulate different scenarios:

- Single master to single slave

- Multiple masters to one slave
- Multiple slaves accessed simultaneously
- Backpressure conditions
- Invalid address cases

Stimulus signals were generated, and outputs were monitored.

6.12 Simulation and Verification

Simulation was performed using EDA tools. Waveforms were analyzed to check:

- VALID/READY handshake correctness
- Proper arbitration
- Correct address decoding
- Proper response generation

The system was verified to be AXI protocol compliant.

7. Working

7.1 Overall Operation

The crossbar operates using the AXI4-Lite protocol, where transactions are divided into independent channels. Whenever a master initiates a transfer, the request passes through the following stages:

Input buffering → Address decoding → Arbitration → Slave communication → Response routing

Each of these stages ensures safe, conflict-free data transfer.

7.2 Write Transaction Working

A write transaction consists of three channels:

Write Address (AW), Write Data (W), and Write Response (B).

Step 1: Master Sends Write Address

The master places the address on the AW channel and asserts AWVALID. The skid buffer stores this address if the crossbar or slave is not ready.

Step 2: Address Decoding

The address decoder compares the incoming address with predefined slave address ranges. It generates a one-hot decode signal selecting the target slave.

If no slave matches, the crossbar prepares an error response.

Step 3: Arbitration

If multiple masters request the same slave, arbitration logic decides which master gets access. A grant signal is given to only one master at a time.

Step 4: Write Data Transfer

The master sends data on the W channel. The skid buffer ensures data is held safely if the slave applies backpressure ($WREADY = 0$).

Step 5: Pending Transaction Tracking

Counters track outstanding write operations to ensure no overflow and to confirm when data is still expected.

Step 6: Slave Response

After completing the write, the slave sends a response on the B channel (OKAY or ERROR).

Step 7: Response Routing

The crossbar routes the response back to the correct master using stored grant/index information.

7.3 Read Transaction Working

A read transaction uses the AR (Read Address) and R (Read Data) channels.

Step 1: Master Sends Read Address

The master places the address on AR channel and asserts ARVALID. The skid buffer stores it if the slave is not ready.

Step 2: Address Decoding

The decoder determines which slave holds the requested address.

Step 3: Arbitration

If multiple masters request the same slave, arbitration selects one master.

Step 4: Address Forwarding

The selected slave receives the read address.

Step 5: Slave Sends Data

The slave sends read data and response (RDATA, RRESP).

Step 6: Response Routing

The crossbar routes the data back to the correct master based on grant signals.

7.4 Role of Skid Buffers

Skid buffers temporarily store transactions during stalls. This prevents data loss and breaks long combinational paths, improving timing.

7.5 Role of Arbitration Logic

Arbitration ensures that:

- Only one master connects to a slave at a time
- Conflicts are avoided
- Fair channel switching is maintained

7.6 Role of Control Logic

Control logic manages:

- Grant release
- Channel switching
- Handshake synchronization
- Stall handling

It ensures smooth data flow across channels.

7.7 Handling Stalls (Backpressure)

If a slave is not ready:

- READY signal goes low
- Skid buffer stores the transaction
- Counters track pending operations

Transfer resumes when READY becomes high.

7.8 Error Handling

If an address does not match any slave:

- Crossbar generates INTERCONNECT_ERROR
- Response is sent back to the master

7.9 Summary of Working

1. Master sends request
2. Skid buffer stores it
3. Address is decoded
4. Arbitration selects master
5. Data is transferred
6. Slave sends response
7. Response routed back

Thus, the crossbar ensures reliable communication between multiple masters and slaves while maintaining AXI protocol rules.

8. List Of Signals

1. Global and Interface Signals

Signal Name	Purpose
S_AXI_ACLK	Global clock signal.
S_AXI_ARESETN	Global active-low reset signal.
S_AXI_AWVALID	Indicates the master has a valid write address.
S_AXI_AWREADY	Output from crossbar indicating it can accept a write address.
S_AXI_AWADDR	The write address bus from the master.
S_AXI_WVALID	Indicates the master has valid write data.
S_AXI_WREADY	Output from crossbar indicating it can accept write data.
S_AXI_WDATA	The write data bus from the master.
S_AXI_WSTRB	Write strobes indicating which byte lanes hold valid data.
S_AXI_BVALID	Indicates a valid write response is available for the master.
S_AXI_BREADY	Indicates the master can accept a write response.
S_AXI_BRESP	The write response status (e.g., OKAY, ERROR).
S_AXI_ARVALID	Indicates the master has a valid read address.
S_AXI_ARREADY	Output from crossbar indicating it can accept a read address.
S_AXI_ARADDR	The read address bus from the master.
S_AXI_RVALID	Indicates the slave has provided valid read data.
S_AXI_RREADY	Indicates the master can accept read data.
S_AXI_RDATA	The read data bus returning to the master.
S_AXI_RRESP	The read response status.
M_AXI_AWADDR	Outgoing write address routed to the selected slave.
M_AXI_WDATA	Outgoing write data routed to the selected slave.
M_AXI_BRESP	Write response received from the slave.

M_AXI_ARADDR	Outgoing read address routed to the selected slave.
M_AXI_RDATA	Read data received from the slave.

2. Internal Arbitration and Routing Signals

Signal Name	Purpose
wrequest / rrequest	Bitmask indicating which slave a master is requesting.
wgrant / rgrant	Bitmask showing which slave is currently granted to each master.
swgrant / srgrant	Indicates if a master currently has any grant active.
mwgrant / mrgrant	Indicates if a slave is currently occupied by a master.
swindex / srindex	The index of the slave assigned to a specific master.
mwindex / mrindex	The index of the master assigned to a specific slave.
wdecode / rdecode	Output from the address decoder indicating the targeted slave bitmask.
stay_on_channel	Logic flag to keep a master-slave connection open during a transaction.
leave_channel	Logic flag to tear down a grant after a stall or idle period.
linger_counter	Timer that delays tearing down a grant to allow back-to-back transfers.
requested_index	The index of the slave a master is currently trying to access.

3. Intermediate Skid Buffer and Decoding Signals

Signal Name	Purpose
skd_awvalid	Valid signal for write address after the master's skid buffer.
skd_awaddr	Buffered write address from the skid buffer.
skd_wvalid	Valid signal for write data after the master's skid buffer.
dcd_awvalid	Valid signal for write address after it has been decoded.
slave_awaccepts	Internal flow control signal determining if a slave can accept a write address.
slave_waccepts	Internal flow control signal determining if a slave can accept write data.
slave_raccepts	Internal flow control signal determining if a slave can accept a read address.

4. Transaction Management and Status Signals

Signal Name	Purpose
awpending	Counter tracking outstanding write address requests.
rpending	Counter tracking outstanding read address requests.
missing_wdata	Tracks how many write addresses have been accepted without matching data.
wdata_expected	Flag indicating the write data channel is waiting for data related to an address.
swfull / srfull	Indicates if the internal transaction counters are at maximum capacity.
swempty / srempty	Indicates if there are no outstanding transactions for a master.

9. Waveform & Simulation

- The following fig-9.1 shows the simulation waveform of parallel write transactions from multiple masters targeting the same slave. It illustrates address and data handshaking, arbitration under contention, and delayed write responses from a slow slave, confirming correct outstanding transaction handling by the AXI-Lite crossbar.

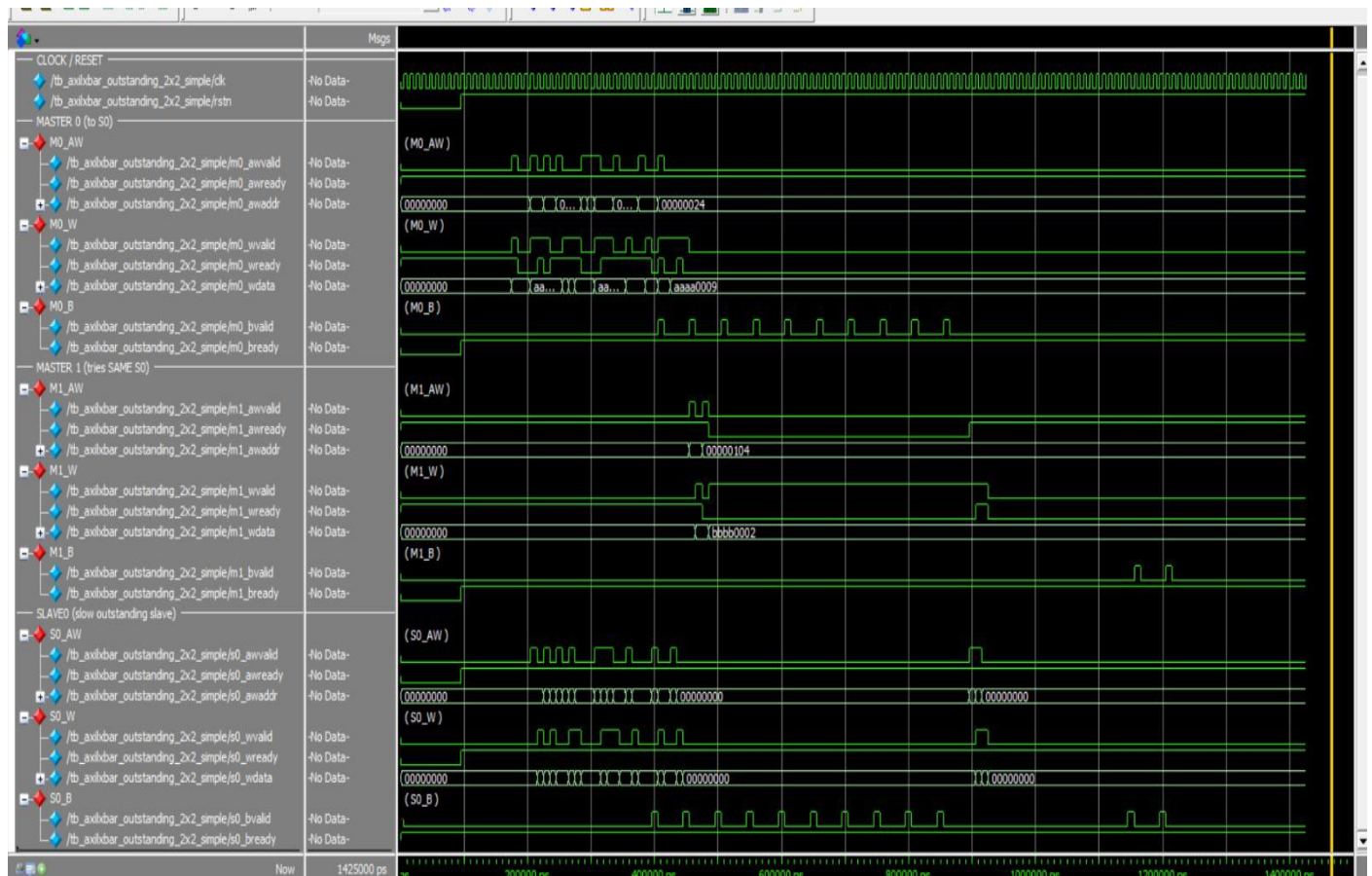


fig No.9.1: Outstanding Write Transaction Handling in AXI-Lite Crossbar

- The following fig-9.2 shows the simulation waveform of AXI-Lite transactions through the designed crossbar. It demonstrates correct address handshaking, data transfer, and response generation for read and write operations, validating proper routing and protocol-compliant behaviour of the crossbar.

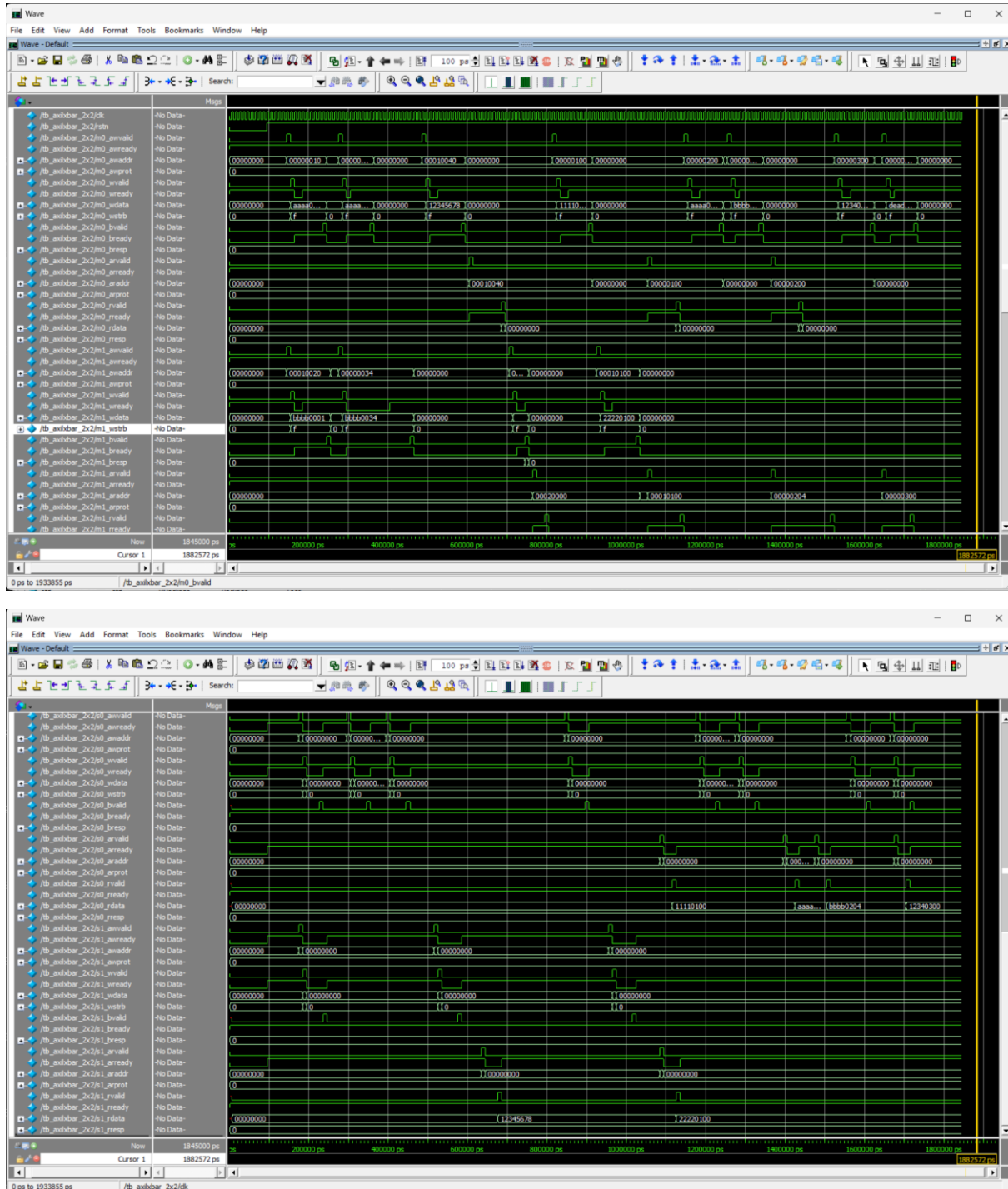


Fig. 9.2: AXI-Lite Crossbar Read and Write Transaction Waveform

- The following fig-9.3 shows the simulation transcript of AXI-Lite transactions through the designed crossbar. It demonstrates correct address handshaking, data transfer, and response generation for read and write operations, validating proper routing and protocol-compliant behaviour of the crossbar.

```

# [975000] XBAR -> S1 W data=2220100
# [1025000] XBAR <- S1 B resp=00
# [1035000] M1 <- B resp=00
# [1045000] M0 -> AR addr=00000100
# [1045000] M1 -> AR addr=00010100
# [1095000] XBAR -> S0 AR addr=00000100
# [1095000] XBAR -> S1 AR addr=00010100
# [1125000] XBAR <- S0 R data=11110100
# [1135000] M0 <- R data=11110100 resp=00
# [1135000] XBAR <- S1 R data=2220100
# [1145000] M1 <- R data=2220100 resp=00
S5 Results: M0 data=11110100 resp=00 | M1 data=2220100 resp=00
---- SCENARIO 6: Read contention (M0->S0, M1->S0) ----
# [1155000] M0 -> AR addr=00000200
# [1155000] M1 -> W data=aaaa0200
# [1165000] XBAR -> S0 AR addr=00000200
# [1195000] XBAR -> S0 W data=aaaa0200
# [1235000] XBAR <- S0 B resp=00
# [1245000] M0 <- B resp=00
# [1245000] M1 -> AR addr=00000204
# [1275000] M0 -> W data=bbbb0204
# [1285000] XBAR -> S0 AR addr=00000204
# [1295000] XBAR -> S0 W data=bbbb0204
# [1335000] XBAR <- S0 B resp=00
# [1345000] M0 <- B resp=00
# [1375000] M1 -> AR addr=00000200
# [1375000] M1 -> AR addr=00000204
# [1405000] XBAR -> S0 AR addr=00000200
# [1435000] XBAR <- S0 R data=aaaa0200
# [1445000] M0 <- R data=aaaa0200 resp=00
# [1495000] XBAR -> S0 AR addr=00000204
# [1515000] XBAR <- S0 R data=bbbb0204
# [1525000] M1 <- R data=bbbb0204 resp=00
S6 Results: M0 data=aaaa0200 resp=00 | M1 data=bbbb0204 resp=00
---- SCENARIO 7: Mixed RW contention on S0 (M0 write, M1 read) ----
# [1535000] M0 -> AR addr=00000300
# [1545000] M0 -> W data=12340300
# [1545000] XBAR -> S0 AR addr=00000300
# [1575000] XBAR -> S0 W data=12340300
# [1615000] XBAR <- S0 B resp=00
# [1625000] M0 <- B resp=00
# [1655000] M0 -> AR addr=00000310
# [1655000] M1 -> AR addr=00000300
# [1665000] M0 -> W data=dead0310
# [1675000] XBAR -> S0 AR addr=00000310
# [1685000] XBAR -> S0 W data=dead0310
# [1695000] XBAR -> S0 AR addr=00000300
# [1715000] XBAR <- S0 R data=12340300
# [1725000] M1 <- R data=12340300 resp=00
# [1725000] XBAR <- S0 B resp=00
# [1735000] M0 <- B resp=00
S7 Results: M1 read data=12340300 resp=00
---- DONE ----

# [155000] M0 -> AR addr=00000010
# [155000] M1 -> AR addr=00010020
# [165000] M0 -> W data=aaaa0001
# [165000] M1 -> W data=bbbb0001
# [185000] XBAR -> S0 AR addr=00000010
# [185000] XBAR -> S1 AR addr=00010020
# [195000] XBAR -> S0 W data=aaaa0001
# [195000] XBAR -> S1 W data=bbbb0001
# [235000] XBAR <- S0 B resp=00
# [245000] M0 <- B resp=00
# [245000] XBAR <- S1 B resp=00
# [255000] M1 <- B resp=00
---- SCENARIO 2: Contention (both masters write S0) ----
# [285000] M0 -> AR addr=00000030
# [285000] M1 -> AR addr=00000034
# [295000] M0 -> W data=aaaa0030
# [295000] M1 -> W data=bbbb0034
# [305000] XBAR -> S0 AR addr=00000030
# [315000] XBAR -> S0 W data=aaaa0030
# [355000] XBAR <- S0 B resp=00
# [365000] M0 <- B resp=00
# [405000] XBAR -> S0 AR addr=00000034
# [415000] XBAR -> S0 W data=bbbb0034
# [455000] XBAR <- S0 B resp=00
# [465000] M1 <- B resp=00
---- SCENARIO 3: Read-after-write correctness ----
# [495000] M0 -> AR addr=00010040
# [505000] M0 -> W data=12345678
# [525000] XBAR -> S1 AR addr=00010040
# [535000] XBAR -> S1 W data=12345678
# [585000] XBAR <- S1 B resp=00
# [595000] M0 <- B resp=00
# [615000] M0 -> AR addr=00010040
# [645000] XBAR -> S1 AR addr=00010040
# [685000] XBAR <- S1 R data=12345678
# [695000] M0 <- R data=12345678 resp=00
---- SCENARIO 4: Illegal address (expect DECERR from xbar) ----
# [715000] M1 -> AR addr=00020000
# [725000] M1 -> W data=deadbeef
# [755000] M1 <- B resp=11
# [775000] M1 -> AR addr=00020000
# [805000] M1 <- R data=00000000 resp=11
---- SCENARIO 5: Parallel reads (M0->S0) and (M1->S1) ----
# [825000] M0 -> AR addr=00000100
# [835000] M0 -> W data=11110100
# [855000] XBAR -> S0 AR addr=00000100
# [865000] XBAR -> S0 W data=11110100
# [905000] XBAR <- S0 B resp=00
# [915000] M0 <- B resp=00
# [935000] M1 -> AR addr=00010100
# [945000] M1 -> W data=2220100
# [945000] XBAR -> S1 AR addr=00010100

```

Fig. 9.3: Simulation Transcript for AXI-Lite Crossbar Read and Write Transactions.

10. Future Scope

Although the designed AXI4-Lite Crossbar Interconnect enables reliable communication between multiple masters and slaves, several enhancements can be made in the future.

The arbitration unit can be improved by implementing advanced arbitration techniques such as round-robin or priority-based scheduling. This will ensure fairness among masters and allow time-critical devices to access shared slaves with reduced latency.

Another enhancement is adding Quality of Service (QoS) control to manage bandwidth allocation. This helps in systems where some peripherals require faster response compared to others.

Power efficiency can be increased by introducing clock gating and low-power control logic, so inactive channels consume minimal power during idle conditions.

The design can be scaled to support a larger number of masters and slaves, making it suitable for complex System-on-Chip (SoC) architectures used in FPGA-based and embedded systems.

Security can be improved by incorporating address protection and access control mechanisms to restrict unauthorized master access to certain slaves.

Further, the design can include error logging and performance monitoring counters to track transaction latency, traffic load, and fault conditions.

Finally, extending verification using UVM-based testbenches and formal verification can increase design robustness and industry-level reliability.

These enhancements will make the AXI4-Lite crossbar more efficient, scalable, secure, and suitable for advanced embedded applications.

11. Conclusion

The AXI4-Lite Crossbar Interconnect designed in this project enables reliable communication between multiple masters and slaves. The system uses address decoding to route transactions, arbitration logic to manage access conflicts, and control logic to maintain proper handshake and data flow. Skid buffers improve timing performance, while pending transaction counters ensure protocol correctness.

Simulation verifies that read and write operations are correctly transferred according to AXI4-Lite rules. Overall, the design provides an organized, efficient, and scalable interconnect solution suitable for FPGA and embedded systems.

12. References

1. Casper-toolflow.readthedocs
2. GitHub - pulp-platform/axi: AXI SystemVerilog synthesizable IP modules and verification infrastructure for high-performance on-chip communication
3. GitHub – depretet /axi-crossbar
4. AMBA AXI Protocol, All About VLSI: Youtube
5. AXI4-Lite Protocol Overview
<https://developer.arm.com/architectures/system-architectures/amba/amba-specifications>
6. AXI4-Lite Interface Tutorial (examples)
https://github.com/muhammadtalhasami/Axi4_lite_interface
7. AXI4-Lite Verification Models (Open Source)
<https://github.com/OSVVM/AXI4>
8. OpenCores AXI Projects
<https://opencores.org/projects/ax4lbr>